

Clasificación de imágenes en Tiny ImageNet

Arquitectura y parámetros de entrenamiento

1. Descripción del problema

Tiny ImageNet es un subconjunto de ImageNet reducido a 200 clases, con imágenes de 64×64 píxeles. El objetivo es entrenar un clasificador que asigne correctamente cada imagen a una de esas 200 categorías, utilizando transfer learning sobre un modelo preentrenado en ImageNet completo.

2. Arquitectura de la red neuronal

2.1 Backbone: ResNet50

Se usó ResNet50 como arquitectura base por dos razones concretas: primero, sus pesos preentrenados con ImageNet (variante IMAGENET1K_V2) ya contienen representaciones visuales de alta calidad que transfieren bien a subconjuntos del mismo dominio; segundo, el bloque residual de ResNet mitiga el problema del gradiente desvaneciente y permite entrenar de forma estable capas más profundas sin degradación.

La red tiene cinco etapas convolucionales con bloques tipo Bottleneck (conv 1×1 → conv 3×3 → conv 1×1). La salida del último bloque produce un mapa de características de 2048 canales que el Average Pooling Global convierte en un vector de 2048 dimensiones.

2.2 Capa de clasificación

Se reemplazó la capa fc original (1000 clases de ImageNet) por una capa lineal fc: Linear(2048 → 200), ajustada al número de clases de Tiny ImageNet. El resto de los pesos del backbone quedaron descongelados para permitir un fine-tuning completo.

3. Preprocesamiento y augmentación de datos

El pipeline de transformaciones distingue entre entrenamiento e inferencia. Para entrenamiento se aplica un Resize a 256 píxeles seguido de un RandomResizedCrop a 224×224, lo que introduce variación de escala y posición en cada paso. Encima de eso se usa RandAugment con dos operaciones y magnitud 9, una técnica que selecciona y combina transformaciones fotométricas y geométricas de forma aleatoria; esto es especialmente útil cuando el conjunto de entrenamiento es relativamente pequeño y el riesgo de sobreajuste es alto.

Para validación y test se usa únicamente Resize(256) seguido de CenterCrop(224), sin ninguna aleatoriedad, para que la evaluación sea reproducible. En todos los casos se normaliza con la media y desviación estándar de ImageNet ([0.485, 0.456, 0.406] y [0.229, 0.224, 0.225]), que coincide con la distribución sobre la que ResNet50 fue originalmente preentrenado.

4. Parámetros de entrenamiento

4.1 Función de pérdida

Se usa Cross-Entropy con label smoothing = 0.1. El label smoothing redistribuye una fracción pequeña de la probabilidad objetivo hacia las clases incorrectas, lo que penaliza al modelo por volverse demasiado confiado y ayuda a generalizar mejor en datasets con cierta ambigüedad visual entre clases.

4.2 Optimizador

Se utiliza Adam con una tasa de aprendizaje inicial de $1e-4$. Este valor es conservador respecto a los defaults típicos de Adam ($1e-3$), lo que es apropiado en fine-tuning porque preserva los pesos preentrenados y evita destruir representaciones útiles en las primeras épocas.

4.3 Scheduler

Se emplea ReduceLROnPlateau monitoreando el `val_loss` con factor de reducción 0.1 y paciencia de 3 épocas. Si la pérdida de validación no mejora en 3 épocas consecutivas, la tasa de aprendizaje se divide entre 10. Esto permite que el modelo explore con una LR alta al inicio y ajuste fino hacia el final sin intervención manual.

4.4 Configuración del DataLoader

Se entrena con batch size de 64 durante 15 épocas, con 4 workers en paralelo y `pin_memory` activado para acelerar la transferencia CPU→GPU. El shuffle está habilitado solo en el loader de entrenamiento.